

DOCKER_TASK_1

➔ Task Title

Docker-01

➔ Objective

The objective of this task is to watch the Docker-01 video posted on slack and worked on executing the initializing Ansible shown in video.

➔ Prerequisites

List all requirements before starting:

- Virtualization
- Containerization
- Infrastructure
- Hypervisor
- Operation system (O.S) – Windows, Linux, Ubuntu
- Bins/libraries
- Applications

➔ Issues Faced: Briefly mention issues and how you fixed them. Posted below in the last.

Conclusion

Conclusion has been provided for every task at the end.

➔ Docker:

Docker is a powerful tool for containerization.

What is containerization? And similar to containerization?

Ans. Alternative tools:

- LXC (Linux container)
- Docker
- Podman

➔ Difference between Containerization and Virtualization?

Virtualization:

- Virtual machines are dependent on operating system
- Virtual machines will take long time to start up & shutdown.
- Virtual machines size is very high and it is not flexible to move from one

→ **environment to another environment.**

Containerization:

- Containers are independent of O.S
- Container will be very fast to start up and to shut down.
- Containers size will be very low making them flexible to move from one

→ **environment to another env.**

Virtualization

App1 App2 App3

bins/libraries bins/libraries bins/libraries

o.s (Ubuntu) Guest o.s (Ubuntu) Guest o.s (Ubuntu) Guest

Hypervisor

Infrastructure

→ **Containerization**

App1 App2 App3

bins/libraries bins/libraries bins/libraries

Containerization

(Docker, Podman, LXC)

Host Operating system

Infrastructure

Infrastructure- CPU, RAM etc.

→ **Hypervisor**- is a software which will us to create virtual machine.

→ **Operating system**: Will help us to deploy our application

→ **O.S in containerization**: We do not require Operating system in

Containerization, instead, we can download the image and test.

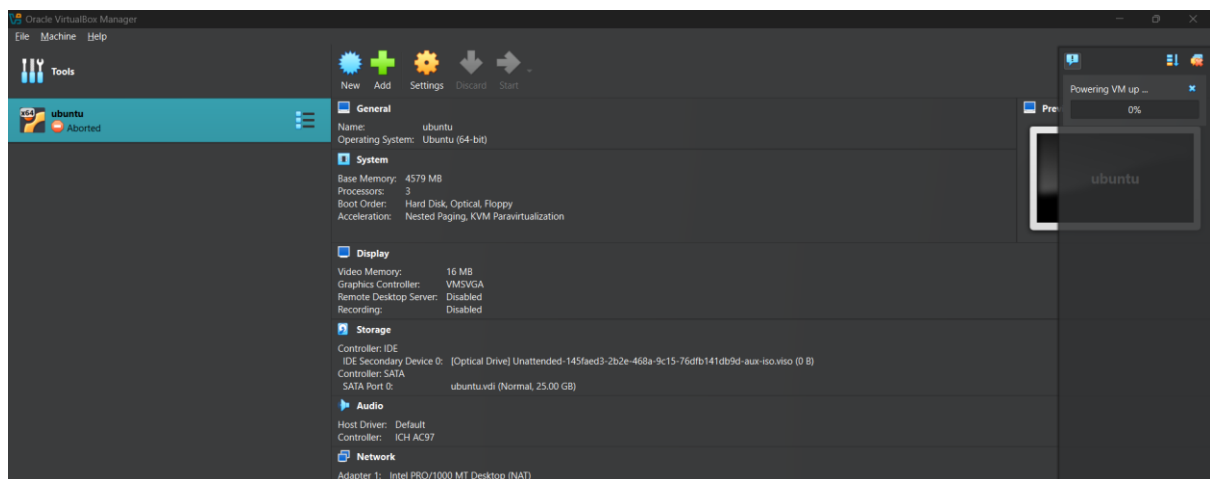
When there is no dependencies between O.S and Containerization, the O.S which has kernels will be shared with bins/libraries.

Tasks:

- ❖ Setup a virtual machine using Ubuntu image.
- ❖ Install docker on ec2.
- ❖ Pull 5 docker images.(nginx,apache tomcat,ubuntu,jenkins,sonarqube)
- ❖ Run nginx container and expose on port 81.
- ❖ Delete the apache tomcat image from local.
- ❖ Inspect the jenkins image,sonarqube image.
- ❖ Run jenkins container and run one sample job.

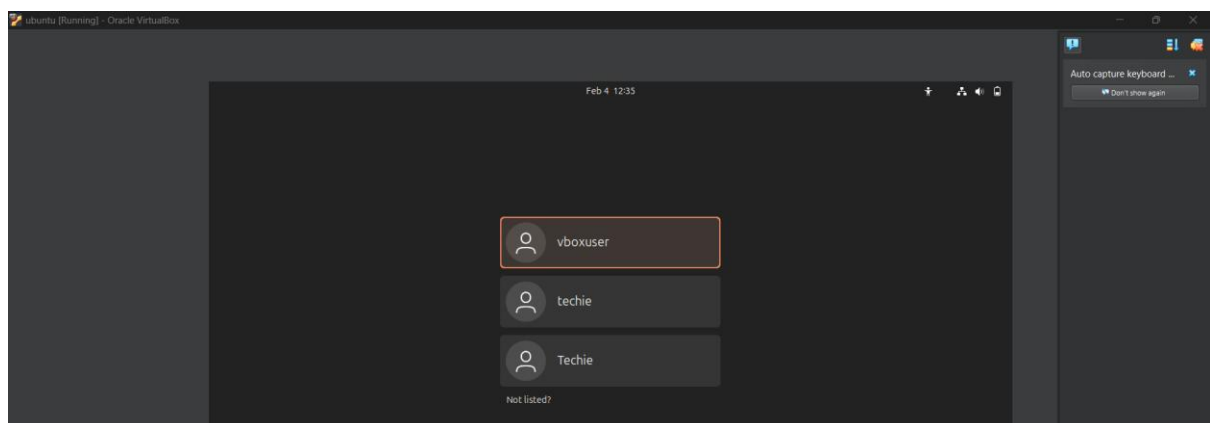
1. Setup a virtual machine using Ubuntu image.

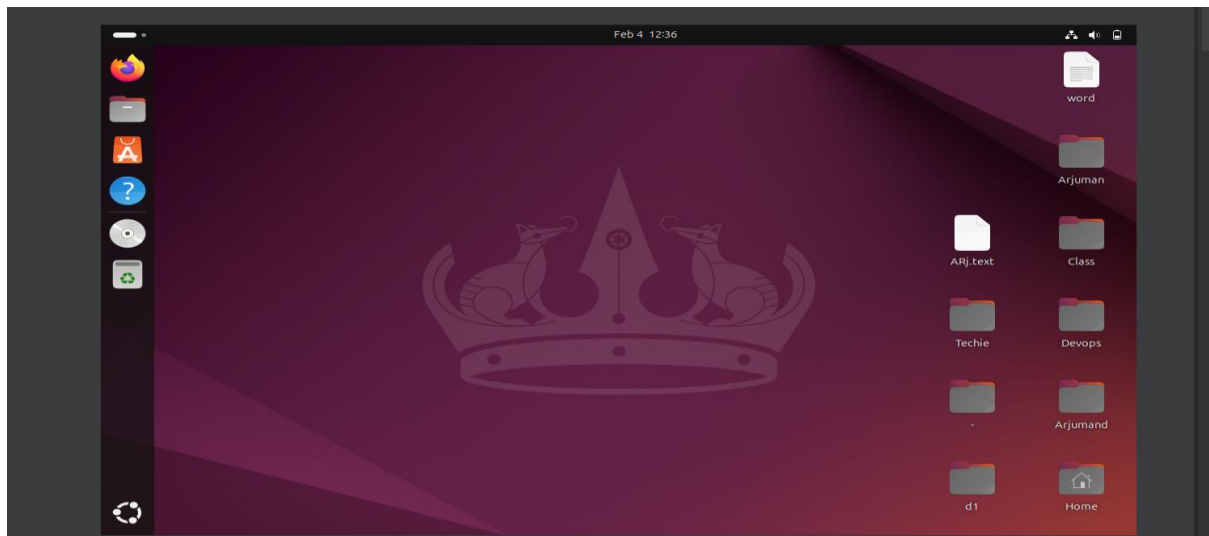
→set up virtual machine



→login to ubuntu

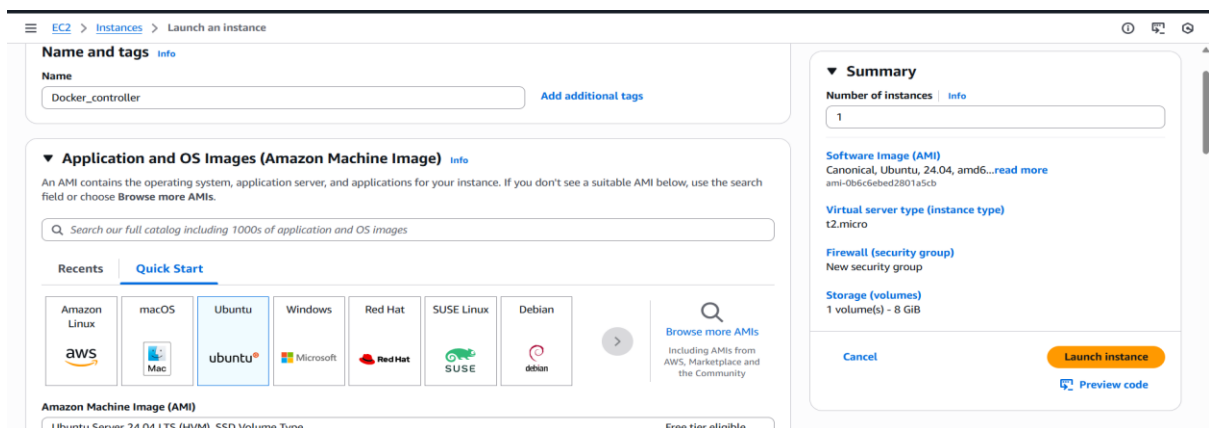
→oprn terminal



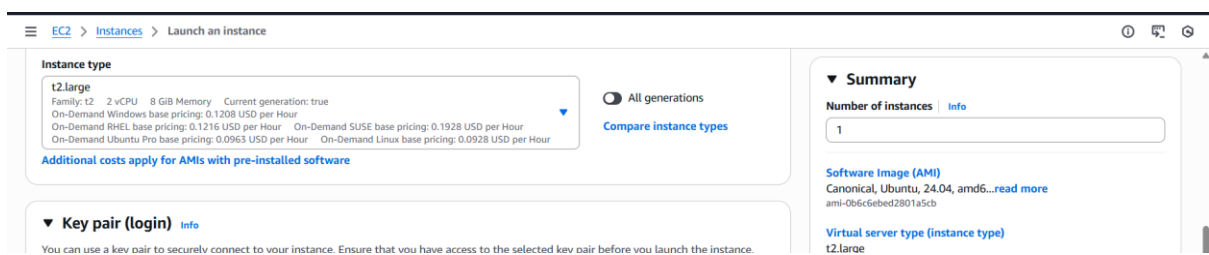


→ Step by step Implementation:

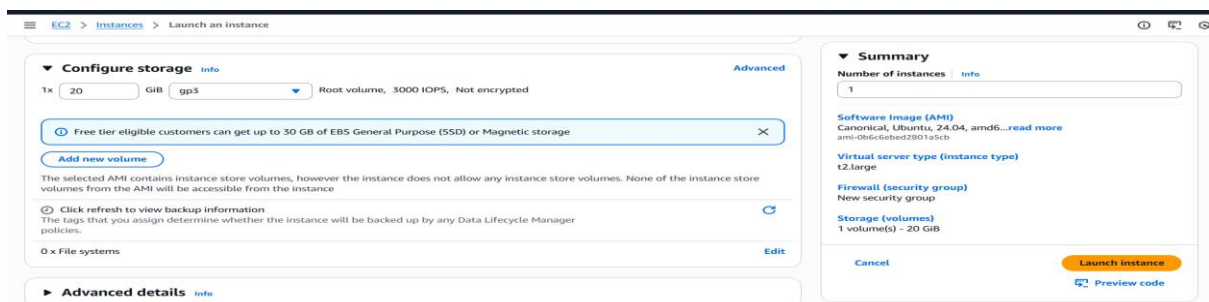
Step 1: Install a virtual machine in ec2 and select Ubuntu image



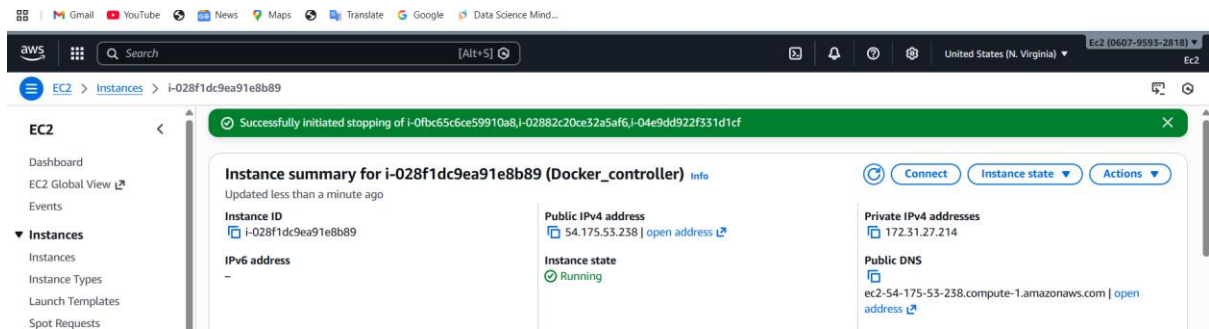
→select t2.large and ubuntu image



→Select 20Gb as volume



→ instance is launched successfully.



2. Install docker on ec2.

→ connect with ec2 using ssh

```
root@ip-172-31-27-214:~# apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
```

→ apt install docker.io -y

```
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 dnsmasq-base amd64 2.90-2ubuntu0.1 [376 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker.io amd64 28.2.2-0ubuntu1-24.04.1 [28.3 MB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 ubuntu-fan all 0.12.16+24.04.1 [34.2 kB]
Fetched 76.1 MB in 1s (76.2 MB/s)
Preconfiguring packages ...
Selecting previously unselected package pigz.
(Reading database ... 71752 files and directories currently installed.)
Preparing to unpack .../0-pigz_2.8-1_amd64.deb ...
Unpacking pigz (2.8-1) ...
Selecting previously unselected package bridge-utils.
Preparing to unpack .../1-bridge-utils_1.7.1-1ubuntu2_amd64.deb ...
Unpacking bridge-utils (1.7.1-1ubuntu2) ...
Selecting previously unselected package runc.
Preparing to unpack .../2-runc_1.3.3-0ubuntu1-24.04.3_amd64.deb ...
Unpacking runc (1.3.3-0ubuntu1-24.04.3) ...
Selecting previously unselected package containerd.
Preparing to unpack .../3-containerd_1.7.28-0ubuntu1-24.04.2_amd64.deb ...
Unpacking containerd (1.7.28-0ubuntu1-24.04.2) ...
Selecting previously unselected package dns-root-data.
Preparing to unpack .../4-dns-root-data_2024071801-ubuntu0.24.04.1_all.deb ...
Unpacking dns-root-data (2024071801-ubuntu0.24.04.1) ...
Selecting previously unselected package dnsmasq-base.
Preparing to unpack .../5-dnsmasq-base_2.90-2ubuntu0.1_amd64.deb ...
Unpacking dnsmasq-base (2.90-2ubuntu0.1) ...
Selecting previously unselected package docker.io.
Preparing to unpack .../6-docker.io_28.2.2-0ubuntu1-24.04.1_amd64.deb ...
Unpacking docker.io (28.2.2-0ubuntu1-24.04.1) ...
```

→ sudo systemctl start docker

→ sudo systemctl enable docker

```
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-27-214:~# sudo systemctl start docker
sudo systemctl enable docker
root@ip-172-31-27-214:~# docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1-24.04.1
root@ip-172-31-27-214:~#
```

i-028f1dc9ea91e8b89 (Docker_controller)

→ Now docker is up and running

```
root@ip-172-31-27-214:~# systemctl enable docker
root@ip-172-31-27-214:~# docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1-24.04.1
root@ip-172-31-27-214:~# systemctl status docker
* docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-02-04 12:10:50 UTC; 3min 5s ago
   TriggeredBy: docker.socket
   Main PID: 2319 (dockerd)
   Tasks: 20 (limit: 2208)
   CPU: 32.4ms
   CGroup: /system.slice/docker.service
           └─ 1 /usr/bin/dockerd --root // --containerd=/run/containerd/containerd.sock

Feb 04 12:10:50 ip-172-31-27-214 dockerd[2319]: time="2026-02-04T12:10:50.541508004Z" level=info msg="Loading containers: start."
Feb 04 12:10:50 ip-172-31-27-214 dockerd[2319]: time="2026-02-04T12:10:50.537531056Z" level=info msg="Loading containers: done."
Feb 04 12:10:50 ip-172-31-27-214 dockerd[2319]: time="2026-02-04T12:10:50.538864722Z" level=info msg="docker daemon commit=28.2.2-0ubuntu1-24.04.1" containerd-anaphe
Feb 04 12:10:50 ip-172-31-27-214 dockerd[2319]: time="2026-02-04T12:10:50.538864722Z" level=info msg="Initializing buildkit"
Feb 04 12:10:50 ip-172-31-27-214 dockerd[2319]: time="2026-02-04T12:10:50.538864722Z" level=warning msg="CDI setup error /etc/cdi: failed to monitor for changes; no su
Feb 04 12:10:50 ip-172-31-27-214 dockerd[2319]: time="2026-02-04T12:10:50.538864722Z" level=warning msg="CDI setup error /var/run/cdi: failed to monitor for changes; no
Feb 04 12:10:50 ip-172-31-27-214 dockerd[2319]: time="2026-02-04T12:10:50.538864722Z" level=info msg="Completed buildkit initialization"
Feb 04 12:10:50 ip-172-31-27-214 dockerd[2319]: time="2026-02-04T12:10:50.538864722Z" level=info msg="Daemon has completed initialization"
Feb 04 12:10:50 ip-172-31-27-214 dockerd[2319]: time="2026-02-04T12:10:50.538864722Z" level=info msg="API listen on /run/docker.sock"
```

3. Pull 5 docker images.(nginx,apache tomcat,ubuntu,jenkins,sonarqube)

→ Step 1: Service is active ◊ let's check few commands below

→ Docker pull nginx ◊ this image has been pulled from Docker Hub

→ Docker pull nginx

```
root@ip-172-31-27-214:~# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
0c8d55a45c0d: Pull complete
173e7a5d3717: Pull complete
2711d25abbb0: Pull complete
899cde133485: Pull complete
8ac398fd80e: Pull complete
bc4d011570c3: Pull complete
1eb69dd8819c: Pull complete
Digest: sha256:9dd288848f4495869f76676e419ae2d767ca99fece2ec37ec0261f9fdaab5204
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

→ docker pull httpd

```
root@ip-172-31-27-214:~# docker pull httpd
Using default tag: latest
latest: Pulling from library/httpd
0c8d55a45c0d: Already exists
fa1db8dd446f: Pull complete
4f4fb700ef54: Pull complete
30973b009bab: Pull complete
85b0d820aa56: Pull complete
996228ca33ab: Pull complete
Digest: sha256:b89c19a390514d6767e8c62f29375d0577190be448f63b24f5f11d6b03f7bf18
Status: Downloaded newer image for httpd:latest
docker.io/library/httpd:latest
root@ip-172-31-27-214:~#
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 54.175.53.238 PrivateIPs: 172.31.27.214

→ docker pull tomcat

```
latest: Pulling from library/tomcat
a3629ac5b9f4: Already exists
bd41d65c3828: Pull complete
8be760c2a9a8: Pull complete
4f4fb700ef54: Pull complete
30702afef09d: Pull complete
a99c7608407b: Pull complete
2ca1459a9031: Pull complete
Digest: sha256:6af1184ab272d631d62a712260ac858010a898b835c088cbe447653cdcl3d97
Status: Downloaded newer image for tomcat:latest
docker.io/library/tomcat:latest
root@ip-172-31-27-214:~#
```

→ docker pull ubuntu

```
latest: Pulling from library/ubuntu
a3629ac5b9f4: Pull complete
Digest: sha256:cd1d8a651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
root@ip-172-31-27-214:~#
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 54.175.53.238 PrivateIPs: 172.31.27.214

→ docker pull Jenkins/jenkins

```
root@ip-172-31-27-214:~# docker pull jenkins/jenkins
Using default tag: latest
latest: Pulling from jenkins/jenkins
53c88f1dfeb7: Pull complete
7006c93f4ecd: Extracting [=====>] 1 32.87MB/58.55MB
0d5e30284279: Download complete
aea1c7bd2c60: Download complete
b904a9e50dc1: Download complete
e7d0072c9676: Download complete
4290ee74c05: Download complete
33e31423140b: Download complete
f304968eed54: Download complete
ff9dc9d0ffb: Download complete
4fe16e51ceec: Download complete
b650ec4dbb1c: Download complete
```

→ docker pull sonarqube

```

docker.io/jenkins/jenkins:latest
root@ip-172-31-27-214:~# docker pull sonarqube
Using default tag: latest
latest: Pulling from library/sonarqube
a3629ac5b9f4: Already exists
220e5f46c255: Extracting [----->] 8.651MB/22.96MB
a3307ab29355: Downloading [----->] 39.39MB/157.8MB
24ebe7eabaf2: Download complete
df8fd45c97cd: Download complete
30d803a38141: Downloading [----->] 102.5MB/913.3MB
b499eabd66bb: Download complete
4f4fb700ef54: Download complete

```

→ all docker images are pulled

```

root@ip-172-31-27-214:~# docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
jenkins/jenkins      latest          64cb03009c23   20 hours ago   487MB
httpd                latest         d2825031e94c   34 hours ago   117MB
nginx                latest         248d232ef351   34 hours ago   161MB
sonarqube            latest         aec0336830ce   5 days ago     1.45GB
ubuntu               latest         493218ed0f40   3 weeks ago    78.1MB
root@ip-172-31-27-214:~#

```

4. Run nginx container and expose on port 81.

→ Step 1: Use command- `docker container run -itd -p 81:80 nginx`

→ `docker run -d -p 81:80 nginx`.

Run

- Creates a new container
- Starts it immediately

👉 If the image is not present, Docker downloads it first.

◆ `-d`

Detached mode

- Runs the container in the background
- Your terminal stays free

Without `-d`, nginx logs would block your terminal.

◆ `-p 81:80`

Port mapping (most important part)

Format:

`-host-port: container-port`

So:

- 81 your machine / server port →
- 80 nginx port inside the container →

Meaning:

When someone accesses port 81 on the host, traffic is sent to port 80 inside the nginx container.

What happens after running this command?

- nginx runs inside a container
- nginx listens on port 80 internally

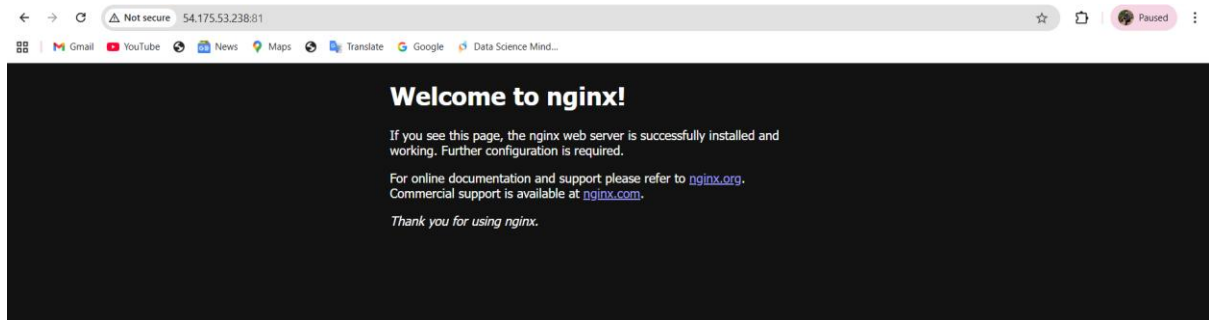
→ Docker exposes it on port 81 externally

```

ubuntu latest 493218ed0f40 3 weeks ago 78.1MB
root@ip-172-31-27-214:~# docker run -d -p 81:80 nginx
214302ae1a6b111d3d8156d875ad521a9301f66804fa60c8f3c96220e215f70d
root@ip-172-31-27-214:~#

```

→ nginx is working on 81 port number



5. Delete the apache tomcat image from local.

→ Step 1: use command- docker ps -a

- Use command- docker images to verify if Tomcat is present

```

root@ip-172-31-27-214:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
0a2405103e9d   tomcat        "catalina.sh run"       7 seconds ago Up 7 seconds  0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   thirsty_zhuker
vsky
670a6bdf5a2c   jenkins/jenkins  "/usr/bin/tini -- /u..." About a minute ago Up About a minute  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 50000/tcp   jolly_ramanuj
an
4050157976aa   jenkins/jenkins  "/usr/bin/tini -- /u..." 4 minutes ago Up 4 minutes  8080/tcp, 50000/tcp   adoring_lisko
y
214302ae1a6b   nginx         "/docker-entrypoint..." 17 minutes ago Up 17 minutes  0.0.0.0:81->80/tcp, [::]:81->80/tcp   infallible_sw
irles
root@ip-172-31-27-214:~#

```

→ if service not started only pulled by the docker then use command “docker rmi httpd”

```

root@ip-172-31-27-214:~# docker rmi httpd
Untagged: httpd:latest
Deleted: sha256:d2825031e94c68c5437ae2f479a1e2b1b2e0a26990c506e17deee19590546735
Deleted: sha256:57838b59415aa1767739fee7c0243fe9251827b1dde0b031abba94db3e25ddbf
Deleted: sha256:64276429bc4d2d47ac9b8a14d99e51611bd5bd1f0502ce318599d2de0bc05ad6
Deleted: sha256:cbcd2515d4aac11bea25255d106f3f90784a4b6f8add3e757ff25f1cf39bf22f
Deleted: sha256:8e1102fabd351ad3ac360a7cf91751be5b4c57f925b54c8a2d41fd6e6f4a8eb3
Deleted: sha256:c14dd8279e21a6c16017e1f03d5fb44e587fd46ad027ae7083322e24ad9c4b7
root@ip-172-31-27-214:~#

```

→ Now tomcat is started so must stop tomcat and then delete “docker stop container-id”

→ Now delete tomcat “docker rm 0a2405103e9d” then “docker rmi tomcat”

```

root@ip-172-31-27-214:~# docker stop 0a2405103e9d
0a2405103e9d
root@ip-172-31-27-214:~# docker rmi tomcat
Error response from daemon: conflict: unable to remove repository reference "tomcat" (must force) - container 0a2405103e9d is using its referenced image fela24e4a16c
root@ip-172-31-27-214:~# docker rm tomcat
Error response from daemon: No such container: tomcat
root@ip-172-31-27-214:~# docker rm 0a2405103e9d
0a2405103e9d
root@ip-172-31-27-214:~# docker rmi tomcat
Untagged: tomcat:latest
Deleted: sha256:fela24e4a16c9df595f3b8afe54a1c2a6d23ccf83be09b919bc2fb91c6390b56
Deleted: sha256:a35aebfcb4887b720b215be1489093672b847c3f579d8229ac910963d8f9093
Deleted: sha256:0cfce2b3f26b07db7c6909372470ba74f61cb3a02737cb4161746e729b3d6
Deleted: sha256:1c146fd53edfde7597a4b378e59e3f46283b2228c884c348001b439f302641
Deleted: sha256:55a7dbb16d051b26c139fd2bf0764a05574b332ba2ac0a74df3960a287ced84
Deleted: sha256:7bdeec6c49307b38e6405f199f00b6b0e91a1c3f2189e7c7f5ce4608c91731b4
Deleted: sha256:229685cb26f3d1db706ac542eee7b7c68ae21e392569b47de3dd7f57e2791437
Deleted: sha256:b29476111056abd42d1f0fd174414af44bceb8632d58e619f12ada13193a89b5
Deleted: sha256:4aba6fa6825957086b4eb801afd7c072e057d1628884457090f6c2d519020e31
root@ip-172-31-27-214:~#

```

6. Inspect the jenkins image,sonarqube image.

→ docker inspect jenkins/Jenkins

→ Step 1: Use command- docker image inspect jenkins:2.30.6


```
root@ip-172-31-27-214:~# docker inspect jenkins/jenkins
[
  {
    "Id": "sha256:64cb03009c233c50be439740447bf71d4b5b1531888f5112e7f4cd91737f4f3",
    "RepoTags": [
      "jenkins/jenkins:latest"
    ],
    "RepoDigests": [
      "jenkins/jenkins@sha256:e9df903e1d9a22132cd26463cfa8cdaa70f53d0dc0834e31f396de687294af22"
    ],
    "Parent": "",
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2026-02-03T16:06:06.357599106Z",
    "DockerVersion": "",
    "Author": "",
    "Config": {
      "User": "jenkins",
      "ExposedPorts": {
        "50000/tcp": {},
        "8080/tcp": {}
      },
      "Env": [
        "PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "JENKINS_HOME=/var/jenkins_home",
        "JENKINS_SLAVE_AGENT_PORT=50000",
        "REF=/usr/share/jenkins/ref"
      ]
    }
  }
]
```

→ docker inspect sonarqube

```
root@ip-172-31-27-214:~# docker inspect sonarqube
[
  {
    "Id": "sha256:aec8336830cadd2c0cb9161fcf3f577b2f0bc0d4b46324d02814385d927dee06",
    "RepoTags": [
      "sonarqube:latest"
    ],
    "RepoDigests": [
      "sonarqube@sha256:1b117276ca3ae9a692b1bd9b75cc28289de958675f217079f8d0a776bef64d8"
    ],
    "Parent": "",
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2026-01-29T21:36:33.672046095Z",
    "DockerVersion": "",
    "Author": "",
    "Config": {
      "User": "sonarqube",
      "ExposedPorts": {
        "9000/tcp": {}
      },
      "Env": [
        "PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "JAVA_HOME=/opt/java/openjdk",
        "LANG=en_US.UTF-8",
        "LANGUAGE=en_US:en"
      ]
    }
  }
]
```

7. Run jenkins container and run one sample job.

→ docker run -d -p 8080:8080 jenkins/Jenkins

→ docker ps

```
670a6bdf5a2c jenkins/jenkins "/usr/bin/tini -- /u..." 22 minutes ago Up 22 minutes 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 50000/tcp jolly_ramanujan
4050157976aa jenkins/jenkins "/usr/bin/tini -- /u..." 26 minutes ago Up 26 minutes 8080/tcp, 50000/tcp adoring_liskov
214302a61a6b nginx "/docker-entrypoint..." 39 minutes ago Up 39 minutes 0.0.0.0:81->80/tcp, [::]:81->80/tcp infallible_swirles
root@ip-172-31-27-214:~#
```

→ jenkins gui and set password and run a job

→ docker exec -it 670a6bdf5a2c cat /var/jenkins_home/secrets/initialAdminPassword

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('First Job') {
6       steps {
7         echo 'My Docker Job'
8       }
9     }
10  }
11 }
```



Jenkins / first_docker_job ▾ / Full Stage View



first_docker_job - Stage View

